Three finite lower bounds for superpermutations

Jay Pantone

October 2026

Abstract

We prove lower bounds of 4,033,379, 6,746,627,228, and 93,891,141,150 for superpermutations on ten, thirteen, and fourteen symbols. The first two follow from finite exclusions in Halfar's model of chain pieces, with a direct passage to unrestricted words. The third uses the small amount of slack in our general lower bound to force all runs into one insertion-loop partition, where their sizes are incompatible with complete coverage. The ten- and fourteen-symbol proofs use Lean's kernel throughout. The thirteen-symbol proof uses `native_decide` for 51 finite computations; the completeness argument and the reduction to words are checked by the kernel.

AI disclosure. The research presented here, the Lean formalization, and this document were generated entirely by AI systems under Jay Pantone's direction. The inherited mathematical work and software retain their original credits. Independent AI review is not human peer review.

Only the ten- and fourteen-symbol results are submitted to Palomar. The thirteen-symbol result is included in this paper for context; its native computation assumptions are outside the submitted proof library.

1 The three bounds

A *superpermutation* on n symbols is a word that contains every permutation of those symbols as a contiguous factor. Let $S(n)$ denote the minimum length of such a word. Our general lower bound $B(n)$ in [1] comes from restrictions on the local structure of a minimum overlap path. There are two ways to improve its value at a fixed n. We can exclude more local configurations by a finite calculation, or we can examine the configurations that would be necessary for equality. We use both methods here.

Theorem 1.1. *We have*

$$S(10) \geq 4{,}033{,}379, \qquad S(13) \geq 6{,}746{,}627{,}228, \qquad S(14) \geq 93{,}891{,}141{,}150.$$

n	General bound $B(n)$	Bound here	Increase
10	4,033,374	4,033,379	5
13	6,746,626,531	6,746,627,228	697
14	93,891,141,149	93,891,141,150	1

Table 1: Comparison with the general theorem of [1].

The finite search framework is due to Halfar [3]. His all-length completeness theorem is essential: searching a collection of short paths alone would not give a bound for arbitrary words. Our contribution for $n = 10$ and $n = 13$ is a stronger parameter certificate and its connection to the

hereditary segment and forest argument of our lower-bound framework. The fourteen-symbol argument is different. Its last step uses the actual occupied rotation classes, information that a list of numerical deficits does not retain.

Halfar's announcement of 6 October 2026 [4] gave $S(10) \geq 4{,}033{,}378$ and $S(14) \geq 93{,}891{,}141{,}008$. The two bounds submitted here exceed those announced values by 1 and 142, respectively. This comparison is with that dated announcement.

2 From words to runs

For permutations u and v of an n-symbol alphabet, the *overlap weight* $w(u, v)$ is n minus the largest length of a suffix of u that is a prefix of v. An ordering of all permutations therefore gives a superpermutation of length n plus its total overlap weight. Conversely, listing the first occurrences of the permutations in any superpermutation gives an overlap path of no greater length. Thus

$$S(n) = n + \min_{P} \operatorname{wt}(P),$$

where the minimum is over Hamiltonian paths in the overlap graph.

Weight-one steps rotate a permutation to the left. The n rotations of a permutation form a *rotation class*. A traversal of all its members is a *block*. If the block is entered at $(x_1, \ldots, x_{n-1}, s)$, it is exited at $(s, x_1, \ldots, x_{n-1})$. A weight-two step to a new rotation class must then enter at

$$(x_2, \ldots, x_{n-1}, x_1, s).$$

The other possible completion of the overlap would revisit the old rotation class. In particular, the final symbol s is unchanged.

A maximal succession of blocks joined by weight-two steps is a *run*. It contains at most $n - 1$ blocks. Its *deficit* is $n - 1$ minus the number of its blocks; a run of deficit zero is *full*. A *light segment* is a succession of runs with weight-three joins. If it has r runs and total deficit D, then it contains $(n - 1)r - D$ rotation classes.

An arbitrary overlap path need not consist of intact blocks. We use the reduced minimum-path decomposition of [2, 1], which extracts block-aligned components and accounts for the remaining vertices. The part of that result needed below is as follows. Suppose that every light segment, and every consecutive factor of such a segment, satisfies

$$D \geq \delta r - \beta, \qquad 0 < \delta < 1, \quad 1 \leq \beta \leq n - 1 - \delta. \tag{1}$$

Put $\alpha = n - 1 - \delta$. There is an integer budget $b \geq 0$ for each reduced minimum path such that

$$(n - 1)! \leq \alpha b + \beta, \qquad \operatorname{wt}(P) \geq (n + 1)(n - 1)! - 3 + b. \tag{2}$$

The forest summation that gives this statement is proved in [1]. The minimum-path decomposition and its local geometric inputs are supplied by [2]. Consequently,

$$S(n) \geq n + (n + 1)(n - 1)! - 3 + \left\lceil \frac{(n - 1)! - \beta}{n - 1 - \delta} \right\rceil. \tag{3}$$

The hereditary condition in (1) is part of the hypothesis; it cannot be replaced by an inequality for one chosen segment.

3 Two finite exclusions

3.1 The model and its completeness theorem

Halfar's model records the entry permutation of each block. A piece is a nonempty list of at most $n-1$ such entries. Consecutive entries in a piece have an overlap of $n-2$ symbols from the exit of the preceding block, and consecutive pieces have an overlap of $n-3$ symbols at their seam. All rotation classes in the entire list are distinct. These conditions give a relaxation of actual light segments. The deficits are again $n-1$ minus the piece lengths.

A *window* begins and ends with positive deficit, and each maximal cluster of positive deficits contains an entry at least two. Write r for the number of pieces and D for the sum of their deficits. We use positive integers q, c, b_0 satisfying $c \le q(n-5)$. The finite exclusion proves

$$q\big(2r + 2(n-4)\big) + cD \le q(n-3)D + b_0 \tag{4}$$

for every model window, of arbitrary length.

Here is the search and the reason that its finite output is enough. Relabelling makes the first entry the identity permutation. At a seam, the next entry is the suffix of length $n-3$ of the preceding exit, followed by a permutation of its first three symbols. There are six possibilities. For each, the search tries each piece length from 1 to $n-1$; the weight-two rule then determines all entries of that piece. A branch is rejected if a rotation class has already appeared.

Set

$$A = q(n-3) - c, \qquad P = AD, \qquad N = 2qr.$$

The state consists of the last exit, the used rotation classes, P, N, and a flag with three possible values. The flag records that the last piece is full, that its positive cluster consists only of ones, or that its positive cluster contains an entry at least two. A violation of (4) is detected by

$$P + b_0 < N + 2q(n-4)$$

when the flag permits a window endpoint.

Two further exclusions make the search practical. A full piece cannot close a cluster consisting only of ones. Also, after a full piece the search need only start a new positive cluster when $P < N$. In a shortest counterexample, a failure of the latter condition would leave a shorter violating suffix. A prefix ending with positive deficit is also discarded when $P - N \ge 2q(n-3)$. A separate finite check excludes a block followed by $n-2$ full pieces. Thus a following string of full pieces has length at most $n-3$, while another positive deficit contributes at least $A - 2q \ge 0$ to $P - N$; these facts justify the surplus cutoff in a shortest counterexample. Halfar's completeness theorem proves these pruning statements together, by assuming a shortest violating window. It proves that a successful finite exclusion implies (4) for all lengths. Its numerical encoding and completeness theorem apply for $5 \le n \le 15$, which includes both alphabet sizes used here.

The search has an explicit recursion allowance. Exhausting it returns failure to certify the exclusion, not success. Thus neither a time limit nor an unexamined branch can establish (4).

3.2 The capacity calculation

The actual run sequence also satisfies three local restrictions: a string of full runs has length at most $n-2$, this drops to $n-3$ when it is adjacent to a nonfull run, and every consecutive deficit pattern $(0, 1, d)$ has $d \ge 2$. These are consequences of the local geometry in [2]. Halfar's list-capacity lemma combines these restrictions with the window inequality on every factor, accounting for the

two endpoints. An endpoint is charged at most $2q(n-2)+b_0$. For every factor of an actual light segment, the result is

$$4qr \le 2AD + 2\big(2q(n-2)+b_0\big).$$

Thus (1) holds with

$$\delta = \frac{2q}{A}, \qquad \beta = \frac{2q(n-2)+b_0}{A}. \tag{5}$$

The passage from an actual segment to the model preserves its entries, rotation classes, and deficits. Applying it to each consecutive factor supplies exactly the hereditary hypothesis required by the forest argument. This avoids an assumption that an arbitrary word is already in the form searched by the program.

The parameters of our two certificates are

n	c	b_0	q	A
10	31	508	127	858
13	86	2100	525	5164.

The corresponding recursion allowances are 614 and 5399. Both certificates include the separate exclusion of a block followed by $n-2$ full pieces. Substitution into (5) gives

n	δ	β	$((n-1)!-\beta)/(n-1-\delta)$
10	127/429	1270/429	77837125/1867
13	525/2582	6825/2582	412260708125/10153.

The ceilings in the last column are 41,692 and 40,604,818. Equation (3) proves the first two claims of Theorem 1.1.

The ten-symbol exclusion is evaluated by Lean's kernel, in smaller subtree certificates. For thirteen symbols, the search is partitioned into 128 buckets at depth 512, arranged in 32 groups. Some groups are split further, for a total of 51 native computations. The bucket index is a rolling hash of the successive piece exits along a search branch. Injectivity is unnecessary: every bucket is checked, and violations or exhausted recursion before the partition depth are retained. A kernel proof establishes that the checks cover the reference search. The additional trust in this result is confined to the evaluations of these 51 computations.

4 Why equality is impossible on fourteen symbols

The remaining improvement comes from the shape of a path that would attain $B(14)$. The argument has three steps. The forest estimate first produces one light segment covering all rotation classes. The deficit inequalities then force every run to have deficit zero or two, with full runs on both sides of each deficit-two run. Finally, the last symbol of the run entry is constant. This places all runs in a single partition of the rotation classes, where a deficit-two run cannot be completed by any other run.

4.1 The slack in the forest estimate

For $n = 14$, the parameters from [1] are

$$p = 36, \qquad t = \frac{569}{15}, \qquad \delta = \frac{1138}{6229}, \qquad \beta = \frac{15816}{6229}, \qquad \alpha = 13 - \delta.$$

Suppose, for a contradiction, that a minimum word has length 93,891,141,149. Choose a reduced minimum overlap path P. Its weight is 93,891,141,135, and its available budget is

$$b_* = \mathrm{wt}(P) - 15 \cdot 13! + 3 = 485{,}829{,}138.$$

The margin in (2) is exceptionally small:

$$\alpha b_* + \beta - 13! = \frac{1398}{6229} < \frac{1}{2}. \tag{6}$$

We retain two kinds of slack that the usual forest summation discards. For any

$$0 < \varepsilon \le \min(1 - \delta, \alpha - \beta),$$

the presence of a component other than the root strengthens the first inequality in (2) by ε. To see this, mark one such component in the forest summation. An unabsorbed component costs at least $\alpha - \beta$. An absorbed component costs at least $1 - \delta$: in the special absorption this is the unused part of its unit charge, while in the other absorption the two boundary deficits are at least two and their sum is at least n. Substitution into the endpoint charge $\max(0, d - \delta)$ gives the same surplus. The marked charge therefore survives summation. This is the forest argument of [1] with one additional charge retained.

The exact weight accounting also has a nonnegative integral residual R, with

$$\mathrm{wt}(P) = 15 \cdot 13! - 3 + b + R.$$

If $R \ge 1$, its contribution to the capacity margin is at least α. Taking $\varepsilon = 1/2$ in (6) therefore forces a single component and $R = 0$. The components of the forest partition all rotation classes, so this one component is complete. It is block-aligned by construction, and its budget is exactly b_*. We use this component for the rest of the argument.

The component has 13! blocks, so it has at least two runs. There is still a possible cost from joins heavier than three and from nonfull endpoint runs. If H is the total excess of the run-boundary weights over three, and the first and last deficits are d_1, d_r, the component inequality retains

$$\max(0, d_1 - \delta) + \max(0, d_r - \delta) + (\alpha - \beta)H.$$

Each nonzero contribution is at least $1/2$. The same margin excludes all three. We have obtained one complete light segment with full endpoint runs, with

$$r = 485{,}829{,}138, \qquad D = 13r - 13! = 88{,}757{,}994. \tag{7}$$

This step is what allows us to use complete coverage at the end of the proof. It would be false to assume such a segment at the outset.

4.2 The deficit sequence

Write the deficit sequence as positive clusters separated by zeros. Let q be the number of clusters, e the number of clusters other than the singleton [2], and L, R the numbers of initial and final zeros. For each internal zero gap u, put

$$S_0 = \sum_u \max(10 - u, 0), \qquad M = \#\{u : u = 11\}.$$

The local restrictions of [1] say that each cluster K has $\sum K \geq |K| + 1$, that $u \leq 11$, and that $L + R \leq 22$. A gap of length 11 between clusters K and J satisfies the stronger inequality

$$11 + |K| + |J| \leq \sum K + \sum J. \tag{8}$$

Here $|K|$ denotes the number of entries in the cluster.

Define its reserve by

$$\Phi(K) = \frac{6229}{15}(\sum K - 2) - \frac{1138}{15}(|K| - 1) - 148\mathbf{1}_{K \neq [2]}, \qquad T = \sum_K \Phi(K).$$

The reserve is nonnegative. If $K \neq [2]$, its mass is at least three, and direct substitution gives

$$\Phi(K) \geq \frac{957}{5}. \tag{9}$$

For example, among the smallest possible exceptional clusters, the least reserve occurs at $[1, 2]$ or $[2, 1]$. Larger mass only increases the bound. At a gap of length 11, equation (8) gives

$$\Phi(K) + \Phi(J) \geq \frac{17207}{5}.$$

Each cluster is incident with at most two gaps, so

$$\frac{17207}{5} M \leq 2T. \tag{10}$$

The chain-packing inequality of [1], with $p = 36$ and $t = 569/15$, gives

$$4q - 148e - \frac{1138}{15} S_0 \leq 144. \tag{11}$$

This is its positive-cluster form: regular clusters contribute four; exceptional clusters and short gaps receive the indicated charges. It applies to every suffix as well as to the whole sequence. We shall retain that suffix information shortly.

The total scalar slack is

$$E = \frac{6229}{15} D - \frac{1138}{15} r + \frac{15816}{15} = \frac{1398}{15},$$

where the last equality follows from (7). Counting the entries in clusters and gaps gives the exact identity

$$\begin{aligned} E = 144 + 148e + \frac{1138}{15} S_0 - 4q + T - \frac{1138}{15} M \\ + \frac{1138}{15}(22 - L - R). \end{aligned} \tag{12}$$

The first group is nonnegative by (11), as is the last term. Equation (10) implies $(1138/15)M \leq T/2$. Hence $E \geq T/2$. If an exceptional cluster existed, equation (9) would give

$$E \geq \frac{957}{10} > \frac{1398}{15},$$

a contradiction. Every positive cluster is therefore [2]. Equation (8) now excludes gaps of length 11.

We need a little more than this classification: each deficit-two run must have at least two full runs on either side. For $u \le 10$, define

$$c(u) = \begin{cases} \dfrac{1138}{15}(10-u) - 144, & u \le 8, \\ 0, & u = 9 \text{ or } 10. \end{cases}$$

All these costs are nonnegative. The suffix form of the packing inequality strengthens (11) to

$$\sum_u c(u) + \frac{1138}{15}(22 - L - R) \le E. \tag{13}$$

For completeness, this follows by a short induction. In each suffix, retain the initial clusters whose intervening gaps have length 9 or 10, and stop at the first shorter gap. Let v be the number of retained clusters minus $569/30$ times the number of 9-gaps among them. The chain-packing inequality gives $v \le 36$. For one cluster, $v = 1$. Prepending a cluster changes v to $1 + v$ across a 10-gap, to $1 + v - 569/30$ across a 9-gap, and to 1 across a gap at most 8. In the last case we use the independent suffix bound $v \le 36$. Induction gives

$$4q - \frac{1138}{15}S_0 + \sum_u c(u) \le 4v \le 144.$$

Substitute this inequality into (12), with $e = T = M = 0$, to obtain (13).

Since $L, R \le 11$, an endpoint with at most nine zeros would contribute at least $2(1138/15) > E$. Thus $L, R \ge 10$. A gap $u \le 6$ would have $c(u) \ge 2392/15 > E$, so all internal gaps are at least seven. In particular, every deficit-two run has the required full runs on both sides.

4.3 A partition that cannot be filled

The *pivot* of a run is the last symbol of its entry permutation. The weight-two rule shows that it is the last symbol of every block entry in the run. We use the following local consequence of the six possible weight-three joins.

Lemma 4.1. *Adjacent full runs in a light segment have the same pivot. In a consecutive triple of runs with deficits* $(2, 0, 0)$*, the first two runs have the same pivot. The reversed assertion holds for* $(0, 0, 2)$*.*

Proof. For adjacent full runs, the chart-forcing lemma of [2] forces the next entry to be the full-run map of the previous entry. This map preserves the last letter. For $(2, 0, 0)$, the full-prefix forcing lemma applies with prefix length two, equal to the preceding deficit. Of the six seam entries, it forces the rotation target, whose last letter is the pivot of the short run. These are respectively the full-run and full-prefix forms of the same local collision calculation. For the third assertion, reverse the segment and every block traversal. A block entered at $(x_1, \dots, x_{13}, s)$ was exited at $(s, x_1, \dots, x_{13})$, so its reversed traversal enters at $(x_{13}, \dots, x_1, s)$. The pivot is unchanged, and the second assertion applies to the reversed triple. The precise local statements and their coordinate calculations are included in [2]. □

Our deficit sequence consists of zeros and isolated twos, and each two has at least two zeros on either side. Lemma 4.1 therefore shows that all runs have a common pivot s.

Fix this symbol s. Deleting s from a rotation class leaves a cyclic ordering of the other thirteen symbols. Conversely, inserting s in each of the thirteen gaps of a fixed cyclic ordering gives thirteen

rotation classes. Call this set an *insertion loop*. These loops partition all rotation classes: the deleted cyclic ordering uniquely determines the loop. For example, deleting 6 and retaining the cyclic ordering 12345 gives a loop of five classes, one for each insertion gap of 6.

Each run with pivot s occupies classes from one such loop. A full run occupies all thirteen classes, and a deficit-two run occupies eleven. Distinct runs occupy disjoint classes. No two runs can therefore belong to the same insertion loop, since each occupies more than half of it. Complete coverage forces each run to fill its loop. Consequently every run is full, which contradicts the positive value of D in (7). Equality at $B(14)$ is impossible. Since lengths are integers, $S(14) \geq B(14) + 1$, as required.

5 The formal statements and their assumptions

The Palomar version of the accompanying repository submits the ten- and fourteen-symbol bounds. Its independent statement files define coverage directly: every list of n distinct letters from `Fin n` must occur as a contiguous factor of the word. They impose no restriction on repeated occurrences, the order of first occurrences, or the decomposition of the word.

The entry points, relative to the repository root, are `Submissions/Ten/Challenge.lean` and `Submissions/Fourteen/Challenge.lean`. Each imports only Mathlib and has a corresponding `Solution.lean` environment proving the same declaration. The common `AxiomCheck.lean` reports the assumptions. Both bounds use only `propext`, `Classical.choice`, and `Quot.sound`.

The thirteen-symbol result is retained in this paper for context, but its native-qualified proof is not included in the Palomar repository or submitted for registration. The original frozen results repository at revision `c9ccb6a0d9ddd1007fdce6efab50807ec47464b9` contains its statement, solution, and the 51 native computation assumptions, with their exact types, in `lean/tools/native_axioms.json`. Its all-length search completeness, partition coverage, and word reduction do not depend on an unproved search heuristic. The 51 computations were also evaluated with Lean 4.33.0; that original complete proof package is pinned to Lean 4.34.1. The additional evaluation is a consistency check and does not remove the native computation assumptions.

The ten-symbol proof is assembled in `proofs/H20/K10.lean`. The fourteen-symbol proof is assembled in `proofs/new_results/Bounds.lean`; its four main ingredients are `ForestRigidity`, `ScalarRigidity`, `PivotRigidity`, and `GlobalPartition`. These are all included. The original thirteen-symbol module, `lean/proofs/H20Native/K13.lean`, remains in the frozen repository only.

References

[1] J. Pantone, *Chain returns and lower bounds for superpermutations*, companion paper in `papers/lower/`.

[2] J. Pantone, *The lower-bound framework for superpermutations*, companion Lean library, namespace `SuperpermutationLowerBound`. See Sections 1–10 and the refinement modules for the minimum-path, forest, and local geometry lemmas.

[3] J. Halfar, *Three lower bounds in Lean: Theorems A, B and C*, lower-bound proof source, revision `cc5c763bd5854317b20dadb138990802656e6f03`, October 2026. The search, completeness, and list-capacity arguments used here are included with attribution in `proofs/LowerBounds/`.

[4] J. Halfar, *New upper bound with 1771/3456 and new lower bounds for $n = 8$ to 14, both in Lean*, announcement to the Superpermutators group, 6 October 2026. https://groups.google.com/g/superpermutators/c/6ozhk3faHDA.